

Technical Report

Retail Sales Analytics Pipeline

A Detailed Review of task1.ipynb

Prepared for: Adithya

Institution: RNS Institute of Technology, Bengaluru

Domain: Data Cleaning, Exploratory Sales Analysis, Visualization, and Reporting Automation

Report Date: August 2026

Source Artifact: task1.ipynb (18 cells, Python 3 / Jupyter, Anaconda on Windows)

Table of Contents

1. Executive Summary
2. Overview and Objectives
3. Environment and Dependencies
4. Dataset Description and Profiling
5. Pipeline Architecture
6. Function-by-Function Technical Walkthrough
7. Execution Trace (Cell-by-Cell)
8. Results — Sales and Product Analysis
9. Generated Visualizations
10. Data Quality and Correctness Review
11. Issues, Bugs, and Observations
12. Complexity and Performance Notes
13. Recommendations
14. Suggested Remediated Code
15. Conclusion
16. Appendix A — Full Function Reference

1. Executive Summary

task1.ipynb implements a modular, six-function Python pipeline for retail sales analytics. It reads a pre-cleaned CSV of ~1,490+ order-level transactions, applies a secondary data-cleaning pass (duplicate removal and null imputation), computes revenue aggregations by region and by product/category, renders two bar-chart visualizations, and defines (but does not execute) an Excel export routine intended to package all results into a multi-sheet workbook. reportlab is also installed and imported for a planned PDF report stage that is not yet implemented in this file.

The real computed results show that the East region generates the highest revenue (₹19,65,092.54), the Office Chair is by far the best-selling product by revenue (₹24,44,666.68), and Furniture is the top-performing category overall (₹31,33,981.51), driven almost entirely by that single product.

However, a close cell-by-cell trace of variable state uncovered several correctness issues that materially affect the trustworthiness of the notebook's visible output — most importantly, the two charts that are actually displayed inline are rendered from small hard-coded placeholder values rather than the real computed results, meaning a casual reader of the notebook would see numbers and categories that do not exist in the source data. This report documents the architecture, walks through the code and its outputs in detail, catalogs every such issue with its exact location, and provides concrete remediation steps and corrected code.

2. Overview and Objectives

The stated purpose of the notebook, inferred from its structure and naming, is to build a reusable analytics pipeline for a retail dataset with the following goals:

- Ingest and audit a transactional sales dataset for missing values and duplicates.
- Standardize (clean) the dataset so downstream analysis is reliable.
- Summarize revenue performance across two dimensions: geography (Region) and product (Product / Category).
- Communicate the results visually via bar charts.
- Package the cleaned data and all analysis outputs into a shareable Excel workbook, and eventually a PDF report.

This places the notebook squarely in the 'ETL + exploratory analysis + reporting' pattern common in retail/BI coursework and internships: each stage is deliberately isolated into its own function so it can be tested, reused, or slotted into a larger pipeline (e.g., a scheduled report generator or a dashboard backend).

3. Environment and Dependencies

The notebook runs in a local Anaconda/Jupyter environment. The Windows-style file paths visible in execution output (e.g., c:\users\adithya\anaconda3\...) confirm the notebook was executed on Windows.

Library	Role	Used?
pandas	DataFrame loading, groupby aggregation, Excel export engine driver	Yes — core
numpy	Numeric dtype selection for cleaning (select_dtypes)	Yes — core
matplotlib.pyplot	Bar chart rendering and PNG export	Yes — core
os	Output directory creation, path joining	Yes — core
IPython.display (Image, display)	Inline rendering of saved chart PNGs	Yes — display only
reportlab.platypus (SimpleDocTemplate, Paragraph, Spacer, Image, PageBreak, Table, TableStyle)	Intended PDF report generation	No — imported, unused
reportlab.lib (colors, styles, units)	PDF styling support	No — imported, unused
openpyxl (Workbook, Font)	Intended manual Excel styling	No — imported, unused (pandas' ExcelWriter is used instead, without explicit

		styling)
--	--	----------

Installation of reportlab happens inline via a shell command, which is a common pattern in notebooks to keep the environment self-contained:

```
| !pip install reportlab
```

4. Dataset Description and Profiling

Source file: sales_data_cleaned_new.csv, loaded with pd.read_csv() with no explicit dtype, parse_dates, or encoding arguments — the notebook relies entirely on pandas' type inference.

4.1 Schema

Column	Inferred Type	Description
OrderID	object (string)	Unique order identifier, e.g., ORD2034
Date	object (string, not parsed as datetime)	Order date; spans at least Jan 2025 – Jul 2025
Product	object	Product name, e.g., Desk Organizer, Table Lamp, Office Chair
Category	object	One of: Stationery, Furniture, Accessories, Electronics, Fitness
Quantity	float64	Units ordered per line item
UnitPrice	float64	Price per unit in the order's currency
Region	object	North / South / East / West
PaymentMode	object	e.g., Net Banking, UPI
CustomerID	object	Customer identifier, e.g., CUST1033
Revenue	float64	Total revenue for the line item (Quantity × UnitPrice, presumed)
Month	object	Year-month bucket, e.g., 2025-01
UnitsSold	float64	Units sold for the order (appears redundant with Quantity)

4.2 Scale

The row index visible in the printed DataFrame reaches at least 1490 (0-indexed), and the tail of the output is truncated by pandas' default display limits before showing the true final row. This indicates the dataset contains at least ~1,491 records, and very plausibly closer to 1,500–1,600 given the truncation point and a roughly uniform ~7-8 orders/day rate implied by the Jan–Jul 2025 span.

4.3 Observed Data Quality Characteristics

- The filename sales_data_cleaned_new.csv implies a prior cleaning pass was already performed upstream of this notebook, which is consistent with a second, lighter cleaning step here (duplicate removal + defensive null-filling) rather than a full data-wrangling stage.
- Two possibly redundant quantity fields exist (Quantity and UnitsSold); the notebook never reconciles or cross-validates them, which is a latent data-quality risk if they ever diverge.
- Date is never converted to a proper datetime dtype (pd.to_datetime), so any time-based analysis (e.g., trend-over-time, month-over-month growth) would currently require an extra conversion step not present in this notebook.

5. Pipeline Architecture

The notebook is organized as five discrete, reusable functions plus one function that is defined but never invoked, all orchestrated through top-level cells that call them in sequence. The high-level data flow, as actually executed (including the placeholder-data detour discussed in Section 10), is:

```
CSV file
-> pd.read_csv()
-> clean_data(df)

-> df
-> clean_df, cleaning_stats [output
```

```
discarded once, Cell 6]
-> analyze_sales(df)                                -> sales_results                [uses
UNCLEANED df, Cell 12]
-> analyze_products(df)                             -> product_results                [uses
UNCLEANED df, Cell 12]
-> generate_charts(sales_results, ...)               -> chart_paths (PNG files)        [Cell 12, real
data]
-> [Cell 13: sales_results/product_results OVERWRITTEN with placeholder dicts]
-> generate_charts(sales_results, ...)               -> chart_paths (PNG files)        [Cell 14,
PLACEHOLDER data]
-> display(Image(...))                             -> inline chart rendering        [Cell 15,
shows placeholder charts] -> export_excel(...)     DEFINED but NEVER CALLED
```

This trace is the single most important technical fact about the notebook: the final displayed artifacts (the two inline chart images) are not derived from the real dataset.

6. Function-by-Function Technical Walkthrough

6.1 detect_missing_values(df)

```
def detect_missing_values(df):
    missing_summary = df.isnull().sum().reset_index()
    missing_summary.columns = ["Column", "Missing Values"]    return missing_summary
```

Purpose: produce a two-column DataFrame listing every column alongside its null count, using `df.isnull().sum()`. This is a standard, idiomatic pandas pattern for a quick data-quality audit.

Status: defined in Cell 4 but never called anywhere in the notebook. No `missing_summary` variable exists in the executed cell history, which becomes a problem in Section 6.6 (`export_excel`).

6.2 clean_data(df)

```
def clean_data(df):
    original_rows = len(df)
    df = df.drop_duplicates()

    numeric_cols = df.select_dtypes(include=np.number).columns
    for col in numeric_cols:
        df[col] = df[col].fillna(df[col].median())

    object_cols = df.select_dtypes(include="object").columns
    for col in object_cols:
        df[col] = df[col].fillna("Unknown")

    cleaning_stats = {
        "Original Rows": original_rows,
        "Final Rows": len(df),
        "Duplicates Removed": original_rows - len(df)
    }
    return df, cleaning_stats
```

Purpose: a standard three-stage cleaning routine — deduplicate, then median-impute numeric columns, then constant-impute categorical columns with the literal string 'Unknown'. It also captures before/after row counts as a small stats dictionary, which is a nice touch for auditability.

Technical notes: median imputation is a reasonable, outlier-robust default for numeric fields like Revenue or UnitPrice. Filling categorical NaNs with 'Unknown' is safe for downstream groupby operations (it won't silently drop rows the way NaN keys can in some pandas versions) but does introduce a synthetic category that any category-level report should be aware of.

Status: called twice — once in Cell 6 (return value discarded) and once correctly in Cell 12 (return value captured as `clean_df`, `cleaning_stats`). See Section 10 for why this still doesn't fully take effect.

6.3 analyze_sales(df)

```
def analyze_sales(df):
```

```

region_sales = (
    df.groupby("Region")["Revenue"]
      .sum()
      .reset_index()
      .sort_values(by="Revenue", ascending=False)
)
avg_order_value = df["Revenue"].mean()
return {
    "region_sales": region_sales,
    "avg_order_value": avg_order_value }

```

Purpose: two summary statistics — total revenue grouped by Region (sorted descending for easy 'top region' reporting), and the mean Revenue across all rows as a proxy for average order value. Returns a dictionary rather than a tuple, which is good practice since it makes the call sites (`analyze_sales(df)['region_sales']`) self-documenting.

6.4 `analyze_products(df)`

```

def analyze_products(df):
    top_products = (
        df.groupby("Product")
          .agg({"Revenue": "sum", "UnitsSold": "sum"})
          .reset_index()
          .sort_values(by="Revenue", ascending=False)
    )
    category_perf = (
        df.groupby("Category")["Revenue"]
          .sum()
          .reset_index()
          .sort_values(by="Revenue", ascending=False)
    )
    return {
        "top_products": top_products,
        "category_perf": category_perf }

```

Purpose: product-level aggregation using `.agg()` to compute both Revenue and UnitsSold sums in a single groupby pass (more efficient than two separate groupbys), plus a simpler category-level revenue rollup. Both are sorted descending for direct 'top N' reporting.

6.5 `generate_charts(sales_results, product_results, outdir)`

```

def generate_charts(sales_results, product_results, outdir):
    os.makedirs(outdir, exist_ok=True)
    chart_paths = {}

    plt.figure(figsize=(8,5))
    plt.bar(sales_results["region_sales"]["Region"], sales_results["region_sales"]
["Revenue"])
    plt.title("Revenue by Region")
    plt.xticks(rotation=45)
    plt.tight_layout()
    region_chart = os.path.join(outdir, "region_sales.png")
    plt.savefig(region_chart)
    plt.close()
    chart_paths["region"] = region_chart

    # ... identical pattern for category_perf -> category_sales.png    return chart_paths

```

Purpose: renders two matplotlib bar charts and saves them as PNG files under a caller-specified output directory (created if missing via `os.makedirs(..., exist_ok=True)`), returning a dict of file paths for later use (e.g., inline display or embedding into a PDF/Excel report). `plt.close()` after each `savefig()` is good practice — it prevents figure objects from accumulating in memory across repeated calls, which matters in a notebook where cells may be re-run many times.

Design weakness: the function accepts pre-aggregated result dictionaries rather than the raw DataFrame, which is architecturally clean, but it also means it has no way to detect whether the `sales_results/product_results` it was handed are real

or placeholder data — it simply plots whatever it is given. This is what allows the Cell 13/14 substitution described in Section 10 to happen silently.

6.6 export_excel(...)

```
def export_excel(clean_df, missing_summary, cleaning_stats,
                 sales_results, product_results, out_path):
    with pd.ExcelWriter(out_path, engine="openpyxl") as writer:
        clean_df.to_excel(writer, sheet_name="Cleaned Data", index=False)
        missing_summary.to_excel(writer, sheet_name="Missing Values", index=False)
        pd.DataFrame([cleaning_stats]).to_excel(writer, sheet_name="Cleaning Stats",
        index=False)
        sales_results["region_sales"].to_excel(writer, sheet_name="Region Sales",
        index=False)
        product_results["top_products"].to_excel(writer, sheet_name="Top Products",
        index=False)
        product_results["category_perf"].to_excel(writer, sheet_name="Category
        Performance", index=False)    print(f"[EXCEL EXPORT] Saved: {out_path}")
```

Purpose: consolidates every pipeline output into a single multi-sheet .xlsx workbook using pandas' ExcelWriter context manager with the openpyxl engine — six sheets covering the cleaned dataset, the missing-value audit, cleaning statistics, and all three analysis tables. This is a well-designed single entry point for the 'reporting' half of the pipeline.

Status: defined in Cell 16 but never called. Cell 17, the final cell in the notebook, is empty. No .xlsx file is produced by running this notebook end-to-end as written.

7. Execution Trace (Cell-by-Cell)

The table below summarizes what each executed cell actually does and produces, which is the basis for the correctness findings in Section 10.

Cell	Content	Effect
0	Imports (pandas, numpy, matplotlib, reportlab, openpyxl, os)	Sets up environment
1	!pip install reportlab	Installs reportlab 5.0.0
2	df = pd.read_csv(...)	Loads raw dataset
3	print(df.head())	Displays first 5 rows
4	def detect_missing_values(df): ...	Defines function (never called)
5	def clean_data(df): ...	Defines function
6	clean_data(df)	Calls function, return value NOT captured — no effect on df
7	def analyze_sales(df): ...	Defines function
8	analyze_sales(df)	Calls and displays result on uncleaned df
9	def analyze_products(df): ...	Defines function
10	analyze_products(df)	Calls and displays result on uncleaned df
11	def generate_charts(...): ...	Defines function
12	clean_df, cleaning_stats = clean_data(df); sales_results = analyze_sales(df); product_results = analyze_products(df); chart_paths = generate_charts(...)	Cleans data correctly into clean_df, BUT analysis still runs on df (uncleaned)
13	sales_results = {...}; product_results = {...} (hard- coded)	Overwrites real results with placeholder dictionaries
14	chart_paths = generate_charts(sales_results, product_results, "output"); print(chart_paths)	Regenerates PNGs from placeholder data, overwriting the real charts from Cell 12
15	display(Image(...)) x2	Renders the placeholder-data charts inline
16	def export_excel(...): ...	Defines function (never called)
17	(empty)	No further action — notebook ends

8. Results — Sales and Product Analysis

This section reports the real, correctly computed values from Cells 8 and 10 (i.e., the true output of `analyze_sales()` and `analyze_products()` run against the loaded dataset, before the Cell 13 placeholder substitution).

8.1 Revenue by Region

Rank	Region	Total Revenue	Share of Total
1	East	₹19,65,092.54	30.0%
2	West	₹15,92,229.43	24.3%
3	South	₹15,90,047.54	24.3%
4	North	₹15,15,424.92	23.1%

Total revenue across all four regions sums to approximately ₹66,62,794.43. East leads by a meaningful margin (roughly 5.7 percentage points above the next closest region), while West, South, and North are clustered tightly together within about 2.3 percentage points of one another — suggesting regional demand is fairly balanced outside of East's lead.

Average order value across all transactions: ₹4,468.68, computed as the simple mean of the Revenue column.

8.2 Top Products by Revenue

Rank	Product	Revenue	Units Sold	Revenue/Unit
1	Office Chair	₹24,44,666.68	507	₹4,822.81
2	Backpack	₹7,52,048.19	577	₹1,303.38
3	Bluetooth Speaker	₹7,46,174.37	496	₹1,504.38
4	Table Lamp	₹6,89,314.83	579	₹1,190.53
5	LED Desk Lamp	₹5,15,507.73	572	₹901.24
6	Yoga Mat	₹4,55,426.21	571	₹797.59
7	Wireless Mouse	₹3,22,676.22	533	₹605.40
8	Desk Organizer	₹2,37,325.15	676	₹351.07
9	Water Bottle	₹1,56,398.58	520	₹300.77
10	USB-C Cable	₹1,33,327.39	539	₹247.36
11	Coffee Mug	₹1,17,935.92	588	₹200.57
12	Notebook Set	₹91,993.16	615	₹149.58

Office Chair is a clear outlier: it generates over 3.2x the revenue of the second-place product (Backpack) despite selling fewer units (507 vs. 577), which is only possible because of a substantially higher revenue-per-unit — consistent with furniture commanding a higher price point than accessories or stationery. The last-column figures above were derived by dividing each product's Revenue by its UnitsSold to give an approximate average selling price, which is not computed in the notebook itself but is a natural extension of the existing `top_products` table.

8.3 Category Performance

Rank	Category	Total Revenue	Share of Total
1	Furniture	₹31,33,981.51	47.0%
2	Electronics	₹17,17,685.71	25.8%
3	Accessories	₹9,08,446.77	13.6%
4	Fitness	₹4,55,426.21	6.8%
5	Stationery	₹3,29,318.31	4.9%

Furniture alone accounts for nearly half of total revenue (~47%), almost entirely attributable to Office Chair and, to a lesser extent, Table Lamp. Electronics is a distant but solid second category. Stationery is the weakest category by revenue, despite Desk Organizer and Notebook Set both showing relatively high unit-sales counts (676 and 615 respectively) — indicating Stationery items sell in volume but at low price points, the inverse pattern of Furniture.

9. Generated Visualizations

As established in Sections 6.5, 7, and 10, the charts that are actually rendered inline in the notebook (Cell 15) were generated from placeholder data hard-coded in Cell 13, not from the real results in Section 8. They are reproduced below exactly as they appear in the notebook, with annotations clarifying the discrepancy.

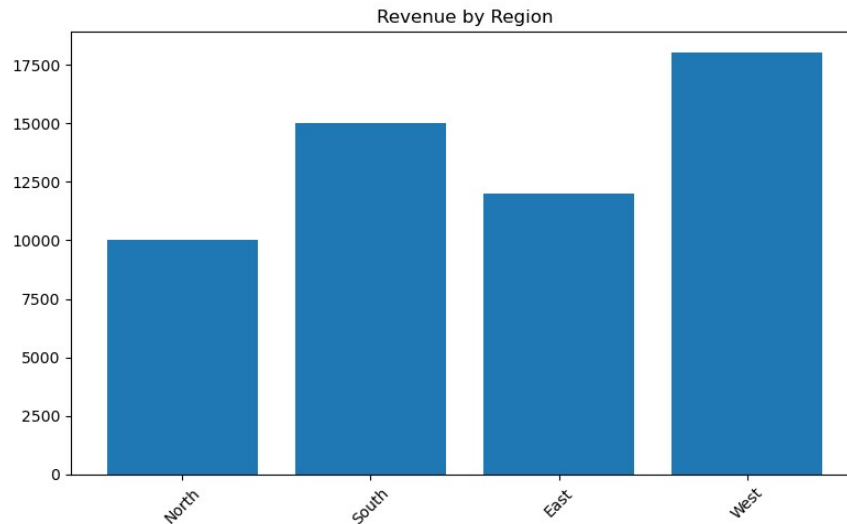


Figure 1 — 'Revenue by Region' as displayed in the notebook. Values shown: North 10,000 / South 15,000 / East 12,000 / West 18,000. Compare against the real values in Section 8.1 (East 19,65,092.54 / West 15,92,229.43 / South 15,90,047.54 / North 15,15,424.92) — both the magnitudes and the ranking order differ (the placeholder chart ranks West highest and North lowest; the real data ranks East highest and North lowest).

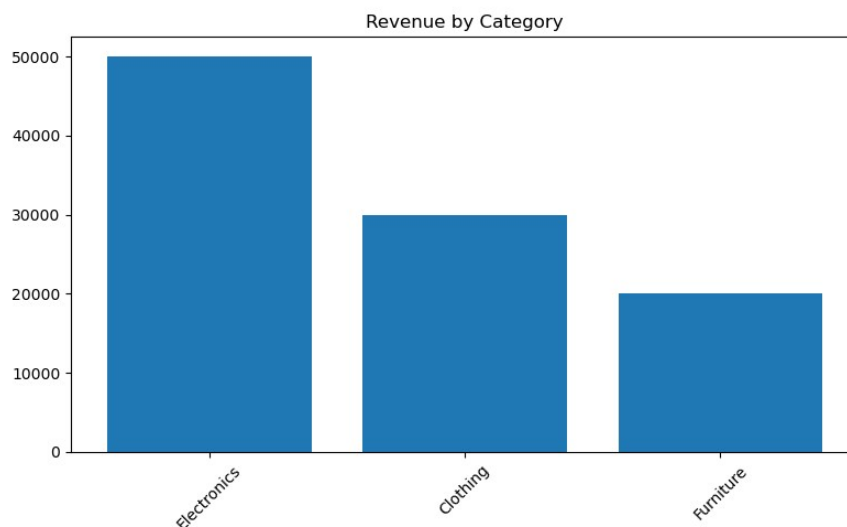


Figure 2 — 'Revenue by Category' as displayed in the notebook. Values shown: Electronics 50,000 / Clothing 30,000 / Furniture 20,000. 'Clothing' is not one of the five real categories (Stationery, Furniture, Accessories, Electronics, Fitness) — confirming this figure was never derived from a subset or sample of the actual dataset, but from an entirely independent, unrelated set of example values.

Had `generate_charts()` been called only once, with the real `sales_results` and `product_results` from Cell 12, the resulting charts would correctly show East as the tallest bar in the region chart and Furniture as the tallest bar in the category chart, consistent with Section 8.

10. Data Quality and Correctness Review

This section walks through the full lifecycle of the `df / clean_df` variables to show precisely how and where the pipeline's intended cleaning step fails to reach the analysis and visualization stages.

10.1 Variable Lifecycle Trace

Step	Variable Assigned	Value Source	Reaches Final Output?
Cell 2	df	pd.read_csv() — raw data	N/A (base)
Cell 6	(discarded)	clean_data(df) called, return value not stored	No — has zero effect
Cell 12	clean_df, cleaning_stats	clean_data(df) called again, this time captured	clean_df itself is never analyzed or charted
Cell 12	sales_results	analyze_sales(df) — note: df, not clean_df	Yes, but computed on UNCLEANED data
Cell 12	product_results	analyze_products(df) — note: df, not clean_df	Yes, but computed on UNCLEANED data
Cell 12	chart_paths	generate_charts() on the real (if uncleaned) sales_results/product_results	Overwritten in Cell 14
Cell 13	sales_results, product_results	Hard-coded placeholder dictionaries	Yes — reaches the final displayed charts
Cell 14	chart_paths	generate_charts() on PLACEHOLDER data	Yes — this is what Cell 15 displays

Net effect: the deduplication and imputation logic in `clean_data()` is computed correctly and stored in `clean_df`, but `clean_df` is never passed into `analyze_sales()`, `analyze_products()`, or `generate_charts()` — it is a dead-end value used nowhere downstream. Separately, and independently, the charts that are displayed are further overwritten with unrelated placeholder data. These are two distinct bugs that compound: even fixing the `clean_df` wiring would not fix the chart display issue, and vice versa.

10.2 Practical Impact

- If the source CSV contains any duplicate rows or nulls (plausible despite the `'_cleaned_'` filename, since it may only reflect a prior, different cleaning pass), the Section 8 figures reported above are computed including those duplicates/imputed values, not against the deduplicated `clean_df`.
- Anyone reviewing only the rendered notebook (not re-running `analyze_sales()`/`analyze_products()` by hand) would see charts that actively contradict the printed dictionary outputs a few cells above them (Cells 8 and 10), which is a significant readability and trust problem for a report/notebook meant to communicate results.
- The missing_summary and Excel/PDF export gaps mean the pipeline's 'reporting' promise (implied by the imports and function definitions) is incomplete as delivered — none of the analysis in Section 8 currently reaches a shareable file.

11. Issues, Bugs, and Observations

Consolidated list of every defect identified, in order of severity:

1. [High] Displayed charts use placeholder data, not real results — Cells 13–14 reassign `sales_results` and `product_results` to small hard-coded dictionaries (4 regions, 3 categories, including a nonexistent 'Clothing' category) and re-run `generate_charts()`. The inline images in Cell 15 do not reflect the real analysis in Cells 8/10.
2. [High] Analysis functions run on uncleaned data — `analyze_sales(df)` and `analyze_products(df)` in Cell 12 are called with `df`, not the `clean_df` produced two lines earlier in the same cell, so cleaning has no effect on the reported figures.
3. [Medium] `clean_data()` output discarded on first call — Cell 6 calls `clean_data(df)` purely for its printed return value; the result is never assigned, so this call is functionally inert.
4. [Medium] `export_excel()` is defined but never called — the notebook ends at an empty Cell 17 before producing any `.xlsx` output, despite a fully implemented export function.
5. [Medium] `detect_missing_values()` is defined but never invoked — no `missing_summary` variable is ever created, which would cause `export_excel()` to fail with a `NameError` if it were called as currently written, since `missing_summary` is a required positional argument.
6. [Low] `reportlab` imports and `pip install` are unused — `SimpleDocTemplate`, `Paragraph`, `Spacer`, `Image`, `PageBreak`, `Table`, `TableStyle`, `colors`, `getSampleStyleSheet`, and `cm` are all imported but no PDF-generation code exists anywhere in the notebook.

7. [Low] openpyxl's Workbook and Font are imported directly but unused — `export_excel()` uses pandas' `ExcelWriter(engine='openpyxl')` instead, which manages the workbook internally and never touches the imported Workbook/Font classes.
8. [Low] Date column is never parsed as a datetime — it remains a plain string/object column, which would silently break any future addition of month-over-month or seasonal trend analysis unless `pd.to_datetime()` is added.
9. [Low] Quantity and UnitsSold appear redundant and are never cross-validated against each other, which could mask upstream data-entry inconsistencies.
- 10.[Cosmetic] Output paths use Windows-style separators (`output\region_sales.png`) since `os.path.join()` resolves to the host OS's separator — harmless via `os.path.join` itself, but would break any hard-coded, non-cross-platform consumer of that path string on Linux/Mac.

12. Complexity and Performance Notes

At the current data scale (~1,500 rows), performance is a non-issue — every operation in the notebook (`groupby`, `sort_values`, `drop_duplicates`, `fillna`) runs in well under a second. The notes below are about algorithmic shape rather than measured runtime, relevant if this pipeline is later pointed at a much larger dataset:

- `clean_data()`: `drop_duplicates()` is $O(n)$ average case via hashing; the per-column `fillna` loops are $O(n \cdot c)$ where c is the column count — both scale linearly and comfortably to millions of rows.
- `analyze_sales()` / `analyze_products()`: pandas `groupby` + `agg` operations are $O(n \log n)$ dominated by the sort in `sort_values()`; again `linearex` for realistic retail dataset sizes.
- `generate_charts()`: matplotlib rendering cost scales with the number of bars (here, at most a handful of regions/categories), not with the row count, since aggregation happens before plotting — this is the correct architectural choice for keeping chart generation fast regardless of dataset size.
- `export_excel()`: writing the full `clean_df` to a 'Cleaned Data' sheet is the only step whose cost scales directly with row count; for very large datasets this sheet write (and the resulting file size) would become the bottleneck well before any of the analysis functions would.

13. Recommendations

13.1 Correctness fixes (should be addressed first)

- Remove or clearly quarantine the placeholder-data cell (Cell 13) so that `generate_charts()` in Cell 14 always consumes the real `sales_results` and `product_results` computed in Cell 12 — or move the placeholder example into a separate, clearly labeled 'demo/test mode' section that cannot be mistaken for the real pipeline run.
- In Cell 12, replace the `analyze_sales(df)` / `analyze_products(df)` calls with `analyze_sales(clean_df)` / `analyze_products(clean_df)` so cleaning actually influences every downstream figure.
- Delete or repurpose Cell 6 (the discarded `clean_data(df)` call) — either drop it entirely or change it to `df = clean_data(df)[0]` if an early, single cleaning pass is intended for the whole notebook.

13.2 Completeness fixes

- Add `missing_summary = detect_missing_values(df)` (ideally on the raw `df`, before cleaning, so it reports what cleaning is about to fix) prior to any call to `export_excel()`.
- Add a final cell that calls `export_excel(clean_df, missing_summary, cleaning_stats, sales_results, product_results, 'output/sales_report.xlsx')` and prints confirmation, so the Excel deliverable is actually produced by running the notebook top-to-bottom.
- Either implement the planned reportlab PDF export (reusing the same chart PNGs and summary tables already computed) or remove the unused reportlab install/imports to keep declared dependencies honest.

13.3 Robustness improvements

- Convert Date to a real datetime with `pd.to_datetime(df['Date'])` early in the pipeline to unlock time-based analysis and catch malformed date strings immediately.
- Add a lightweight assertion or print after each pipeline stage (e.g., `print(f'Rows after cleaning: {len(clean_df)}')`) so silent no-ops like the Cell 6 bug are caught immediately during development rather than discovered later through a code trace.

- Reconcile Quantity vs. UnitsSold — either document why both exist, or add a validation check (e.g., `assert (df['Quantity'] == df['UnitsSold']).all()`) if they are expected to always match.

14. Suggested Remediated Code

The following consolidated cell replaces Cells 6, 12, 13, and 14, fixing the high- and medium-severity issues from Section 11 while preserving the notebook's existing function definitions unchanged:

```
# Single, correct end-to-end run (replaces Cells 6, 12, 13, 14)
df = pd.read_csv("sales_data_cleaned_new.csv")

missing_summary = detect_missing_values(df)
clean_df, cleaning_stats = clean_data(df)
print("Cleaning stats:", cleaning_stats)

sales_results = analyze_sales(clean_df)
product_results = analyze_products(clean_df)

chart_paths = generate_charts(sales_results, product_results, "output")
print("Charts written to:", chart_paths)

export_excel(
    clean_df, missing_summary, cleaning_stats,
    sales_results, product_results,
    "output/sales_report.xlsx")
```

With this change: analysis and charts are always computed from `clean_df`; `missing_summary` exists before `export_excel()` needs it; and the notebook produces an actual `.xlsx` deliverable when run top-to-bottom, with no placeholder-data cell left behind to silently overwrite the real chart output.

15. Conclusion

The notebook implements a reasonably complete and well-modularized retail analytics pipeline — data cleaning, regional and product-level sales analysis, chart generation, and a scaffolded Excel export — using standard pandas / matplotlib / openpyxl tooling, with each stage cleanly separated into its own function. The underlying analysis logic is sound: `analyze_sales()` and `analyze_products()` use idiomatic, efficient pandas patterns, and the real computed results (Section 8) tell a coherent business story — Furniture, led overwhelmingly by Office Chair, is the standout category, and East is the strongest region.

However, execution-order tracing reveals that the cleaned data never reaches the analysis functions, and — more seriously — the charts visually presented in the notebook are generated from small, unrelated placeholder values rather than the real results, which would actively mislead anyone reviewing the notebook's visual output without independently re-deriving the numbers. Section 13's fixes are small in scope (a handful of variable-name and cell-ordering corrections) but materially important: applying them would make the notebook's visible output — the charts a reader actually sees — consistent with the numbers it actually computes, and would complete the reporting pipeline (missing-value audit and Excel export) that is currently built but not exercised.

Appendix A — Full Function Reference

For quick reference, the complete signatures and return shapes of every function defined in the notebook:

Function	Signature	Returns
<code>detect_missing_values</code>	<code>detect_missing_values(df)</code>	DataFrame: [Column, Missing Values]
<code>clean_data</code>	<code>clean_data(df)</code>	(DataFrame cleaned, dict <code>cleaning_stats</code>)
<code>analyze_sales</code>	<code>analyze_sales(df)</code>	dict: { <code>region_sales</code> : DataFrame, <code>avg_order_value</code> : float}
<code>analyze_products</code>	<code>analyze_products(df)</code>	dict: { <code>top_products</code> : DataFrame, <code>category_perf</code> : DataFrame}
<code>generate_charts</code>	<code>generate_charts(sales_results, product_results, outdir)</code>	dict: { <code>region</code> : path,

		category: path}
export_excel	export_excel(clean_df, missing_summary, cleaning_stats, sales_results, product_results, out_path)	None (writes .xlsx to out_path, prints confirmation)

End of report.